Error Handling

Error handling is **not just try–catch** — it's about **building fault-tolerant systems**.
A senior MERN developer treats errors as **data + signal**, not accidents.

An **error** is when the program **cannot continue normally**.

Types of Errors

Type	When Happens	Example
Syntax Error	Code is invalid	const a = ;
Reference Error	Variable not defined	x + 5
Type Error	Wrong data usage	"5".map()
Runtime Error	Happens while running	API failed
Logical Error	Code runs but wrong result	Wrong formula

2. JavaScript Error Object (Internal Structure)

Every error is actually an object:

```
{
  name: "TypeError",
  message: "x is not a function",
  stack: "Where error happened"
}
```

You can access them:

```
try {
  x();
} catch (err) {
```

```
  console.log(err.name);
  console.log(err.message);
  console.log(err.stack);
}
```

👉 stack is extremely important for debugging production apps.

3. try...catch (Execution Control)

Used when you want to **prevent crash** and handle failure gracefully.

```
try {
  const data = JSON.parse(userInput);
} catch (err) {
  console.log("Invalid JSON");
}
```

Without try-catch → app crashes.
With try-catch → app recovers.

4. throw (Creating Custom Errors)

Professionals don't rely only on system errors.
They create **domain-specific errors**.

```
function withdraw(balance, amount) {
  if (amount > balance) {
    throw new Error("Insufficient Balance");
  }
  return balance - amount;
}
```

Now the error has **business meaning**, not just a crash.

5. Custom Error Classes (Used in Production APIs)

In large apps, we define structured errors.

```
class AppError extends Error {
  constructor(message, statusCode) {
```

```
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
  }
}
```

Use it:

```
throw new AppError("User Not Found", 404);
```

Why?

Because production systems must distinguish:

Error Type	Meaning
Operational Error	Expected (bad input, 404, validation)
Programming Error	Bug in code (undefined, null crash)

Only **operational errors** should be shown to users.

6. Async Error Handling (Most Important in MERN)

try-catch **does NOT work automatically** with async unless you use await.

❌ Wrong:

```
try {
  fetchData(); // won't catch
} catch {}
```

✅ Correct:

```
try {
  await fetchData();
} catch (err) {
  console.log(err.message);
}
```

7. Promise Error Handling

Every promise has .catch().

```
getUser()
  .then(user => console.log(user))
  .catch(err => console.log("Failed:", err.message));
```

Error Handling Philosophy (Senior-Level Thinking)

A strong backend follows:

✓ Fail Fast

Detect errors early.

✓ Never Trust Input

Validate everything.

✓ Log Everything

You cannot fix what you don't record.

✓ Don’t Leak Internal Errors to Users

Users see:

Something went wrong

Logs see:

MongoNetworkTimeoutError at user.service.js:42

Q.Your API crashes when MongoDB goes down. How do you handle it?”

What interviewer checks: Do you handle infrastructure failures?

Strong Answer Approach:

- Wrap DB connection with retry / fail-fast logic
- Log error
- Gracefully shut down server (don’t run in broken state)

Q. **“User sends invalid JSON and your server crashes. What do you do?”**

Expected Concept: Validation + Operational Errors

Use validation middleware and return **400 Bad Request**, not crash.

Q. **“How do you avoid writing try-catch in every controller?”**

They want: Async Wrapper Pattern

```
const asyncHandler = fn =>
  (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
```

Q. Difference Between Operational Error and Programming Error?”

Interview Tip:
Operational errors → handled invalid input , db timeout
Programming errors → logged + fixed (don’t hide bugs) undefined var, wrong logic

Q. **“How Do You Handle Errors in Async/Await vs Promises?”**

Expected Explanation:

- async/await → use try-catch
- promises → use .catch()
- In Express → pass error to next()

Q. **What Happens if You Don’t Handle unhandledRejection?”**

Node may:

- Leave app in inconsistent state
- Memory leaks
- Silent failures

Q. **How Would You Debug Production Errors?"**

Expected answer:

- Use logging (Winston / Morgan / console)
- Add request IDs
- Check stack trace
- Reproduce locally
- Never debug directly in production

Q. "Frontend Gets 500 Sometimes — How Do You Trace It?"

Correct Debug Flow:

Check Logs →
Check API Route →
Check DB Query →
Check Input →
Reproduce →
Fix Root Cause

Not guessing. **Trace-driven debugging.**

Q. "What Is the Role of Logging in Error Handling?"

Error handling without logging = useless.

You must log:

- **error message**
- **stack trace**
- **route**
- **user id (if exists)**
- **timestamp**

Q. **How Would You Prevent One Failed Request From Crashing Entire Server?"**

By isolating request lifecycle.

Express already does this if errors are passed to middleware instead of thrown globally.

Most Asked Rapid-Fire Questions

Q: Can try-catch handle async errors without await?
👉 No.

Q: Should you handle every error?
👉 No. Only operational ones.

Q: Where should error handling live?
👉 Central middleware.

Q: What is worst anti-pattern?
👉 Empty catch blocks.

Q: Why not return 200 with error message?
👉 Breaks API semantics.

What is finally?

finally is a block that **ALWAYS executes**, whether:

- ✅ Error happened
- ✅ No error happened
- ✅ You returned early
- ✅ You threw another error

It is used for **cleanup logic**.

```
try {
  // risky code
} catch (err) {
```

```
  // handle error
} finally {
  // always runs
}
```

Think of it as:

“No matter what happens above — run this before leaving.”

🔹 Why finally Exists (Real Reason)

In production, we must **release resources**:

- Close DB connection
- Stop loaders
- Unlock files
- Clear timers
- End transactions
- Hide UI spinner (frontend)

If we don’t → memory leaks, locked resources, hanging requests

☑ finally runs even if you return

```
function test() {
  try {
    return "from try";
  } finally {
    console.log("finally runs");
  }
}

test();
```

Output:

finally runs

Return is delayed until finally finishes.

✅ finally runs even if error is not caught

```
try {
  throw new Error("boom");
} finally {
  console.log("cleanup done");
}
```

The error still throws, but cleanup happens first.

catch → handles the failure
finally → restores the system

🔥 One-Line Interview Answer

"finally is used for deterministic cleanup — code that must execute regardless of success or failure, typically to release resources or reset state."